lorvaq

Живые схемы real-time мониторинга · глаза для серверов маленькой студии

Обзор продукта · 10.10.2026 · подробный дизайн: `docs/design/2026-10-10-core-and-ci-scene.md`

Как читать. Раздел 1 — что такое lorvaq, как выглядит первая сцена и откуда берутся данные. Раздел 2 — принятые решения. Раздел 3 — задачи-пакеты первого проекта «ядро + сцена CI» в порядке зависимости. Пара к kalysta: kalysta — руки (восстановить, переключить, впустить), lorvaq — глаза (увидеть, что происходит и почему).

1. Картина

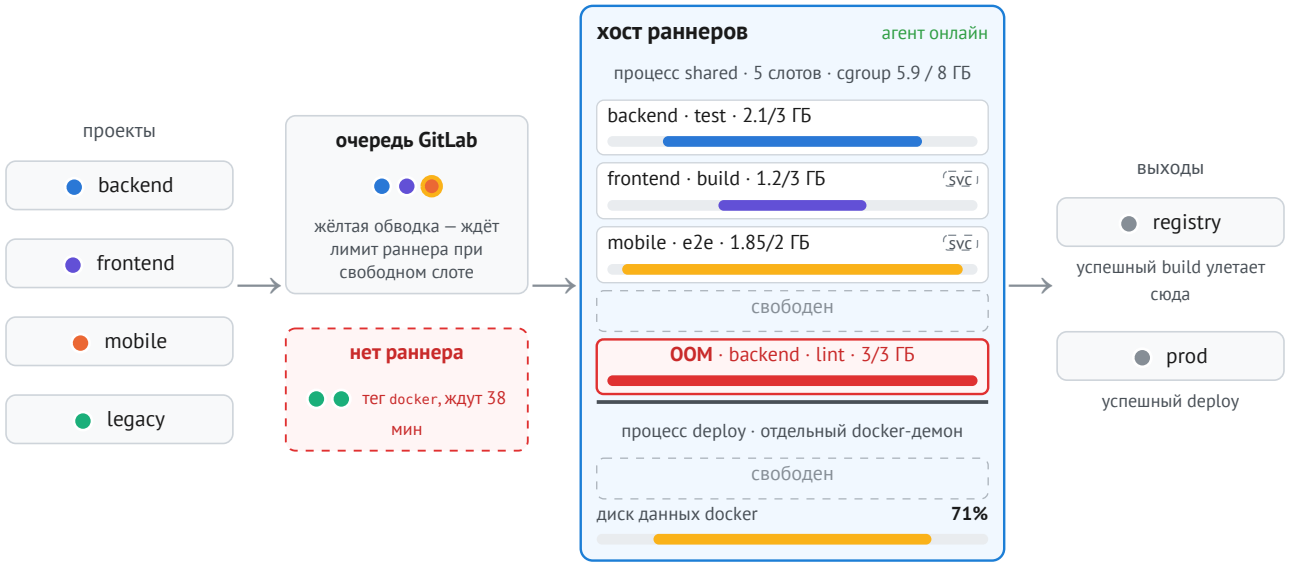
Что это. Самостоятельное self-hosted приложение, которое показывает систему не числами на осях, как Grafana, а **вещами в местах, которые движутся**: джоб едет из очереди в слот раннера, бэкап — с диска в S3, запрос — через сервисы. Цель — с одного взгляда видеть не «сколько», а «почему и где»: этот джоб ждёт, потому что занят лимит раннера; этот застрял — нет раннера с его тегом; этот слот упал по памяти.

Как живёт. Один docker-образ со своим интерфейсом. Сцены встроены и включаются в настройках — каждый берёт нужные. GitLab подключается кнопкой Authorize, хосты — командой с join-токеном; через env только адрес базы и ключ шифрования. Только чтение: lorvaq ничего не запускает и не меняет. Встраивающая система (админка студии) только запускает образ и ставит ссылку.

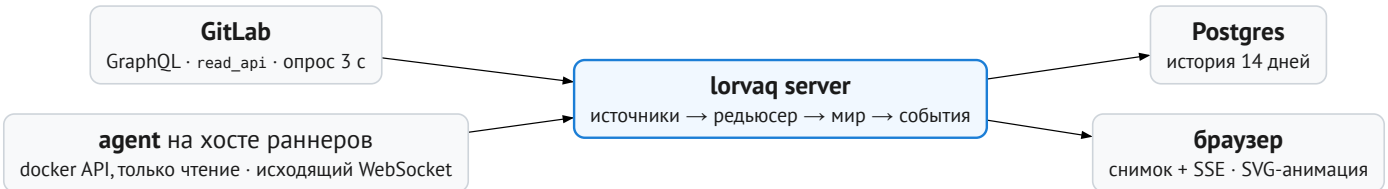
Модель мира. Любая сцена описывается четырьмя понятиями — ядро хранит их и не знает, откуда пришли данные:

	Сцена CI (первая)	Сцена бэкапов	Сцена трафика
Места	проекты, очередь, хосты, процессы раннеров, слоты, выходы	серверы, базы, диск, S3-бакеты	вход, сервисы, базы
Сущности	джобы	прогоны бэкапа	запросы
Перемещения	очередь ⟶ слот ⟶ registry / prod	база ⟶ диск ⟶ S3	вход ⟶ сервис ⟶ база
Уровни	память джоба / лимит, диск хоста	размер бакета, свежесть	латентность, ошибки

Как выглядит сцена CI (симуляция, данные выдуманы; вживую токены едут, слоты наполняются, OOM вспыхивает):



Откуда данные. Сервер опрашивает GitLab и принимает отчёты агентов, сводит их в мир сцены, пишет историю и отдаёт браузеру снимок и поток событий.



2. Что решено

Решение	Почему
Публичный репо `alfinkly/lorvaq`, MIT	продукт автора; любая студия, включая первую, пользуется им по лицензии как внешней зависимостью
Rust, один бинарник: `server` / `agent`	как kalysta: статическая сборка, агент почти не ест память рядом с раннерами
Docker-образ, Postgres (`sqlx`)	запуск одной командой (compose в репо); историю можно читать и из Grafana
Сцены встроены, включаются в настройках	проще плагинов; внешние сцены — когда их начнут писать другие люди
Настройка из интерфейса	GitLab — OAuth Authorize или токен по готовой ссылке; хосты — команда с join-токеном
Один администратор + режимы зрителей	`open` / `proxy` (за forward-auth студии) / `admin`; первый пароль — по коду из лога
Ничего о конкретной студии в репо	ни хостов, ни доменов, ни скриншотов с реальными данными — только `--demo`; `gitleaks` в CI

3. Задачи первого проекта: ядро + сцена CI

СДЕЛАНО Л0 Дизайн

ЧТО ПОСТРОИТЬ модель мира, компоненты, сцена CI, доступ, API, хранение, деградация, тесты — в `docs/design/`.
ГОТОВО, КОГДА дизайн прочитан и принят владельцем; дальше — план реализации по шагам.

НЕ НАЧАТО Л1 Разведка на живом раннере

ЧТО ПОСТРОИТЬ проверить docker-метки gitlab-runner, формат имён контейнеров (`concurrent-N`), события `die`/`OOMKilled`, `statvfs` в rootless-демоне.
РЕШИТЬ на чём держится связка «джоб ↔ слот ↔ память», если меток не хватит.
ГОТОВО, КОГДА записка с фактами (без данных студии) и синтетические фикстуры для тестов агента.

НЕ НАЧАТО Л2 Ядро: мир, шина, снимок и SSE, история

ЧТО ПОСТРОИТЬ типы мира и событий, применение, снимок + события = тот же мир, SSE с `Last-Event-ID` и `410`, Postgres с миграциями и чисткой, живая схема без базы.
ГОТОВО, КОГДА тесты ядра зелёные; сервер в `--demo` отдаёт снимок и поток событий.

НЕ НАЧАТО Л3 Доступ и настройки

ЧТО ПОСТРОИТЬ первый запуск по коду, администратор, сессии, режимы зрителей, шифрование секретов, страницы настроек.
ГОТОВО, КОГДА без входа администратора настройки недоступны в любом режиме; секреты не возвращаются API.

НЕ НАЧАТО Л4 Источник GitLab

ЧТО ПОСТРОИТЬ мастер подключения (OAuth + PKCE с обновлением токена или токен по ссылке), проверка прав, опрос GraphQL, баннер «данные устарели».
ГОТОВО, КОГДА подключение за минуту из интерфейса; отзыв токена виден в настройках.

НЕ НАЧАТО Л5 Агент

ЧТО ПОСТРОИТЬ `lorvaq agent --join`: слоты, память джоба и сервисов, диск, OOM; исходящий WebSocket с переподключением; «Добавить хост» в интерфейсе.
ГОТОВО, КОГДА хост появляется сам после `docker run`; офлайн-агент — серый хост, а не ошибка.

НЕ НАЧАТО Л6 Сцена CI и движок отрисовки

ЧТО ПОСТРОИТЬ редьюсер (очередь, «нет раннера», «ждёт лимит», OOM, выходы) и раскладка на примитивах: контейнер, слот, токен, перелёт, бак, вспышка.
ГОТОВО, КОГДА на реальном GitLab джобы видны с задержкой не больше 5 с; на `--demo` — так же, как на картинке выше.

НЕ НАЧАТО Л7 Релиз

ЧТО ПОСТРОИТЬ GitHub Actions (тесты, clippy, образ в ghcr по тегу), `compose.yml` и `compose.demo.yml`, README на английском с демо-гифкой.
ГОТОВО, КОГДА `docker compose -f compose.demo.yml up` у постороннего человека показывает живую схему.